PONTIFÍCIA
UNIVERSIDADE
CATÓLICA DE CAMPINAS

Outubro de 2026

Um Aplicativo Móvel para Gerenciamento de Treinos com Detecção da Qualidade de Execução de Exercícios via Visão Computacional

Orientador
Antonio Paulo Salgado Forster

Aluno
Fernando Bordin Correa

TCC

Sumário

I Introdução 3

II Objetivos 4

III Revisão Tecnológica 5

IV Arquitetura do Sistema 8

V Detecção de Execução de Exercícios 12

VI Implantação em Nuvem 15

VII Metodologia de Avaliação 16

VIII Resultados e Limitações 17

IX Trabalhos Futuros 19

X Conclusão 20

Introdução

O Problema

- Execução incorreta de exercícios resistidos é causa comum de **lesão** e de **baixa efetividade** do treino

- Praticantes iniciantes raramente têm um **profissional supervisionando cada repetição**

- A avaliação hoje depende de autoavaliação ou de vídeos genéricos, que **não analisam a execução específica do usuário**

- A popularização de câmeras de boa qualidade em *smartphones* e o amadurecimento de bibliotecas de estimativa de pose em tempo real tornaram viável um sistema acessível, **sem sensores vestíveis**

Objetivos

Objetivo Geral

- Desenvolver e avaliar um sistema mobile completo de apoio ao treino de musculação, com um módulo de detecção automática da qualidade de execução baseado em visão computacional

Objetivos Específicos

- App móvel Android (React Native/Expo) para planejamento, execução e histórico de treinos
- Backend que centralize a persistência de dados e hospede o serviço de análise de vídeo
- Pipeline de extração de características a partir de pose humana + classificador correto/incorreto para mais de um exercício
- Implantação em um ambiente de nuvem, validada fora do ambiente de desenvolvimento local
- Identificar e documentar as limitações do modelo atual, propondo direções para superá-las

Revisão Tecnológica

Sensores Inerciais — Velloso et al. (2013)

- 4 IMUs (acelerômetro + giroscópio + magnetômetro) fixadas em luva, bracelete, cinto lombar e halter
- Rosca bíceps unilateral, **5 classes**: 1 execução correta + 4 erros comuns
- Random Forest: **98,03% de acurácia** em validação cruzada 10-fold
- Limitação prática: exige **instrumentar o usuário** com múltiplos sensores — inviabiliza o uso espontâneo numa academia comum

Visão Computacional

- Dispensa qualquer instrumentação além de uma câmera comum
- OpenPose (*bottom-up*, 18/25 pontos) vs. **MediaPipe/BlazePose** (33 pontos, até 75× mais rápido em CPU móvel) — usado neste trabalho
- Posição de câmera também afeta a estimativa de pose (Aguilar-Ortega et al., 2023): diferença **estatisticamente significativa** entre ângulos de câmera

Revisão Tecnológica

Pose Trainer — Chen e Yang (2020)

- OpenPose + regras geométricas manuais + comparação por *Dynamic Time Warping* (DTW) / kNN contra trajetórias de referência
- 4 exercícios com halteres; F1-score entre **0,73 e 1,00**
- **Origem do dataset de treino usado neste trabalho** (elevação frontal e rosca bíceps)

Trabalhos Recentes (2021+)

- Parmar, Gharat e Rhodin (2022): avaliação *self-supervised* de erro de forma, direto do vídeo, sem regras manuais
- Dittakavi et al. (2022) — Pose Tutor: sistema **explicável**, aponta qual articulação causou o erro

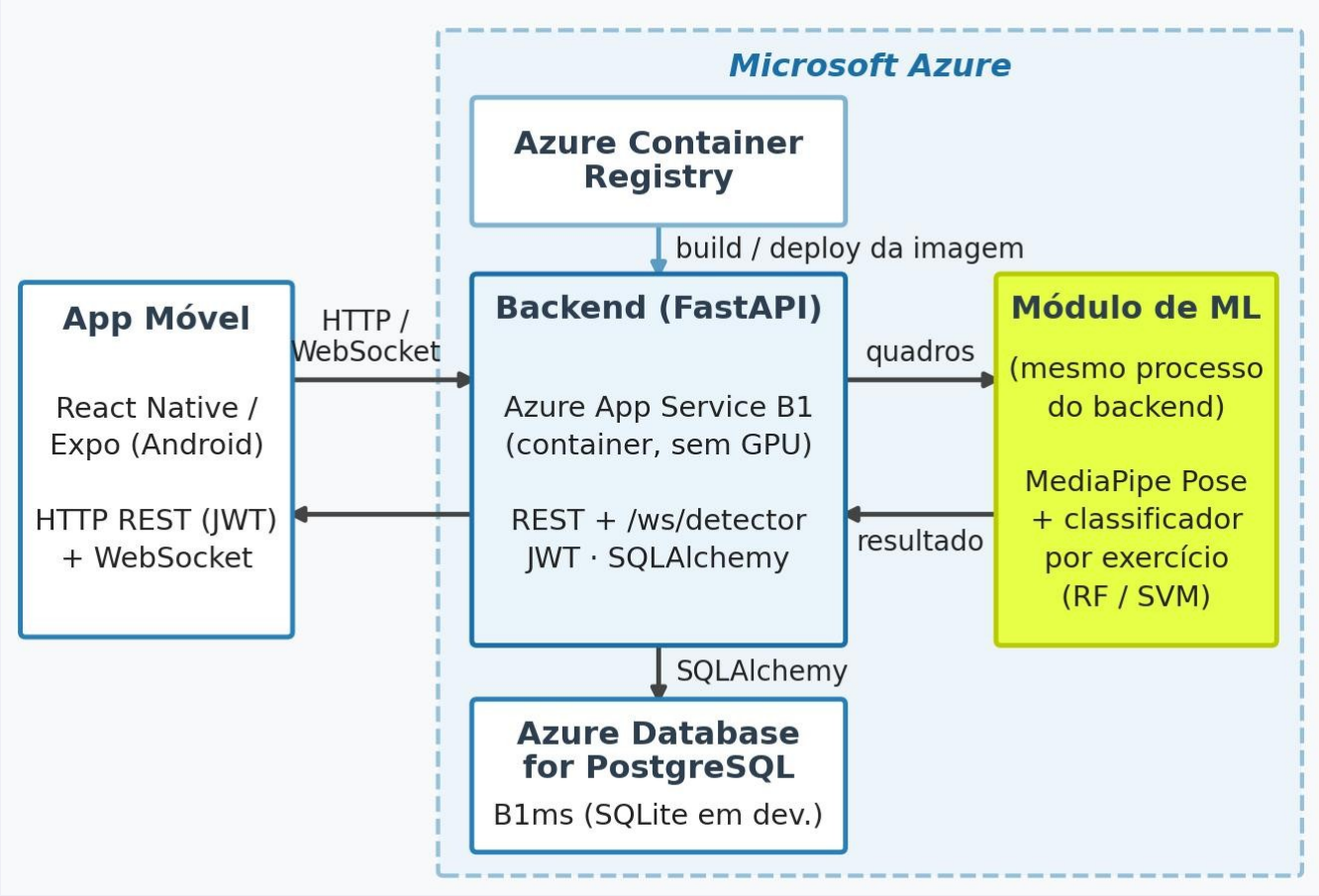
Posicionamento deste Trabalho

Eixo	Velloso et al.	Pose Trainer	Este trabalho
Sensor	IMU vestível	Câmera de smartphone	Câmera de smartphone
Estratégia de decisão	Classificador sobre séries temporais	Regras + DTW/kNN	Classificador (RF/SVM) sobre o quadro de pico
Feedback ao usuário	Classe de erro (A–E)	Qual erro geométrico	Só correto/incorreto + confiança
Origem do dataset	Coletado pelos autores	Coletado pelos autores	Reaproveitado de terceiros (Pose Trainer)
Restrição computacional	Não se aplica	Ambiente de pesquisa	Nuvem sem GPU (Azure App Service B1)

Tabela completa com 8 eixos de comparação no artigo (Seção 2.4).

Arquitetura do Sistema

- 3 componentes: **app móvel** (cliente), **backend** (HTTP + WebSocket) e **módulo de ML** — executado no mesmo processo do backend, não como serviço separado



Aplicativo Móvel

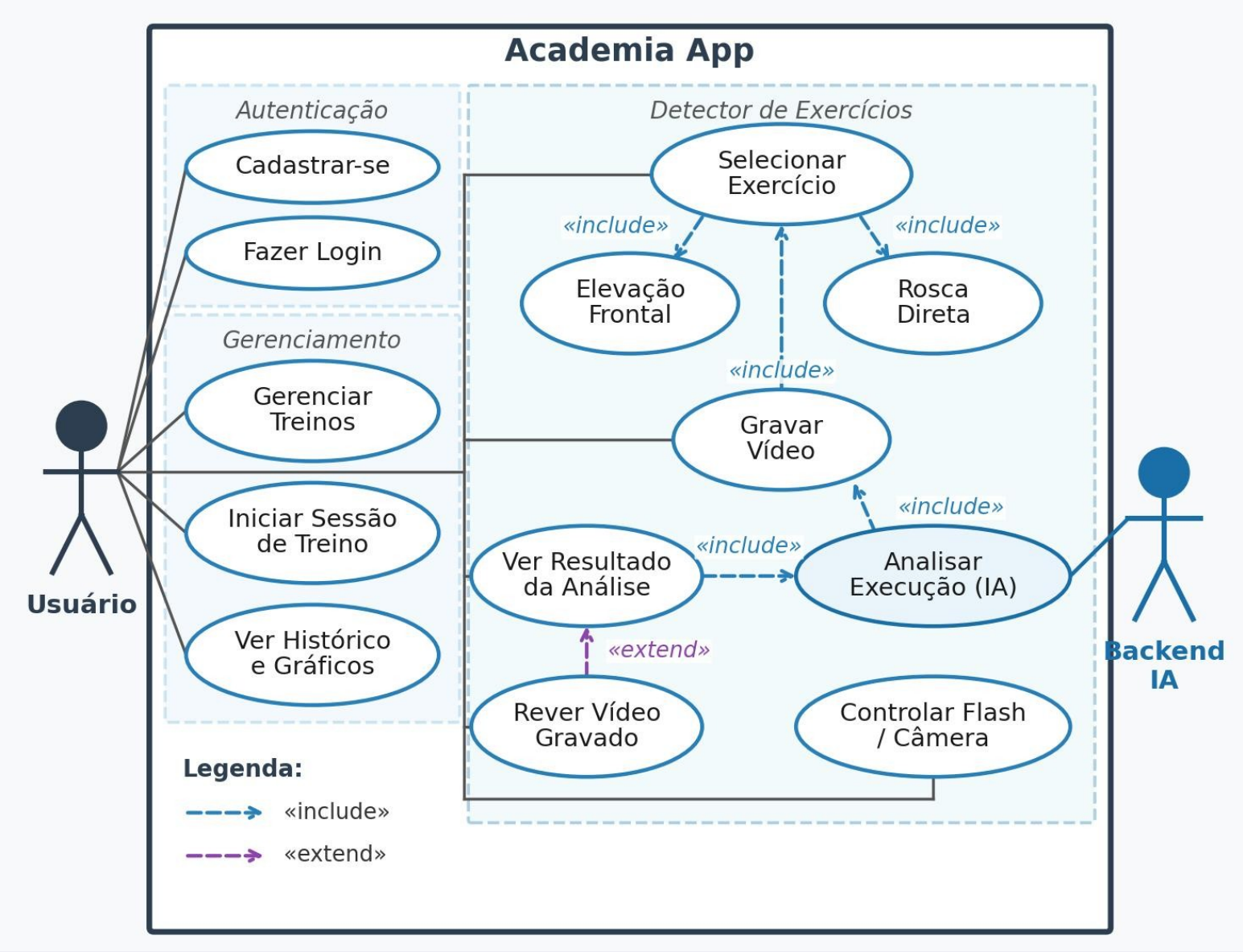
React Native + Expo (Android)

- Autenticação de usuário
- Cadastro de exercícios e montagem de fichas de treino (séries, repetições, carga)
- Execução de sessões com cronômetro e estimativa de calorias
- Histórico e estatísticas mensais de treino
- Tela do detector: grava ou seleciona um vídeo para análise de execução

Comunicação com o Backend

- HTTP (API REST, *token* JWT) e, no detector, também WebSocket

Diagrama de Casos de Uso



Backend

- **FastAPI** (Python) — rotas assíncronas (relevante para o processamento de vídeo) + documentação automática
- **SQLAlchemy** como ORM — SQLite em desenvolvimento, **PostgreSQL em produção**; migrações com Alembic
- Autenticação com **JWT** (*python-jose*) e senhas com *hash* **bcrypt**
- Modelo de dados: usuários, exercícios, treinos e sessões — cada série com o registro real de peso/repetições executados

Detecção de Execução de Exercícios

Estimativa de Pose e Extração de Características

- Cada quadro processado com **MediaPipe Pose** (via OpenCV) → 33 pontos-chave 2D
- Ângulos articulares + alturas relativas do punho, **normalizados** pela distância ombro–quadril do próprio usuário
- Cada exercício tem seu próprio conjunto de características e sua própria regra de "repouso" (delimita início/fim de cada repetição)

Características por Exercício

Exercício	Características
Elevação Frontal	7 — ângulo de braço e cotovelo (cada lado), altura de punho, assimetria
Rosca Bíceps	3 — ângulo de cotovelo, ângulo de braço, altura de punho (lado mais visível)

Detecção de Execução de Exercícios

Classificação Supervisionada por Exercício

- Inferência: classifica o **quadro de pico** da repetição — o de maior ângulo de braço (elevação frontal) ou maior flexão de cotovelo (rosca bíceps)
- **Elevação Frontal:** Random Forest fixo
- **Rosca Bíceps:** escolha automática entre Random Forest, Gradient Boosting e SVM (validação cruzada 5-fold, F1 macro) → modelo atual é um **SVM com kernel RBF**

Quadro de pico, formalmente:

$$\textbf{quadro classificado} = \arg\max_t i_t$$

i_t = métrica de intensidade do exercício no quadro t da repetição

Dataset de Treino

- Dataset público do projeto **Pose Trainer**, extraído via OpenPose (formato BODY_18)
- Cada quadro válido do vídeo vira uma linha independente — **não há segmentação por repetição** no treino

Exercício	Vídeos (bom/ruim)	Amostras (linhas)	Correto/Incorreto
Elevação Frontal	15 / 13	626	343 / 283
Rosca Bíceps	6 / 10	1158	465 / 693

- Vídeos com execuções rotuladas como "correta" ou "incorreta" (ex.: flexão do cotovelo na elevação frontal, uso de momento do corpo na rosca bíceps)

Implantação em Nuvem

- Backend conteinerizado com **Docker** e implantado na **Microsoft Azure**:
  - Azure Container Registry — armazena a imagem
  - Azure App Service, camada B1 — executa o container (sem GPU)
  - Azure Database for PostgreSQL, camada B1ms — persistência
- App móvel compilado como pacote Android autônomo (**EAS Build**, perfil *preview*), já apontando para o backend na nuvem

Metodologia de Avaliação

- Particionamento treino/teste (80/20) e validação cruzada 5-fold feitos **por quadro**, não por vídeo
- O identificador do vídeo de origem **não é preservado** no CSV — impossível garantir que quadros de um mesmo vídeo fiquem só de um lado da partição
- **Métricas reportadas:** acurácia (teste + validação cruzada) para os dois exercícios; F1 macro (validação cruzada) adicional na rosca bíceps

Resultados

- Sistema completo implantado e funcional, testado a partir de um *build* Android real, fora do ambiente de desenvolvimento

Exercício	Acurácia (teste)	Validação Cruzada	Modelo
Elevação Frontal	79%	83% ± 3% (acurácia)	Random Forest
Rosca Bíceps	85%	F1 macro 0,85	SVM (kernel RBF)

Limitações

- Acurácias são um **limite superior otimista**: *split* por quadro (não por vídeo) provavelmente vaza entre treino e teste
- **Mecanismo causal central:** a inferência sempre classifica o quadro de maior amplitude da repetição, mas o treino não seleciona picos — dataset cobre só até ~110°, execuções reais chegam a 172°
- Observação preliminar **não controlada**: câmera operada por terceiro erra mais que autorregistro (plausibilidade externa em Aguilar-Ortega et al., 2023)
- Descasamento de limiar de repouso na rosca bíceps (155° na curadoria do dataset vs. 130° em produção)
- Sem matriz de confusão nem métricas por classe; dataset da rosca bíceps desbalanceado (465/693)
- Segmentação de repetições (detecção de repouso) nunca avaliada isoladamente

Trabalhos Futuros

- **Metodologia de avaliação:** particionamento por vídeo (GroupKFold); alinhar amostragem de treino ao quadro de pico; unificar limiares de repouso; avaliar a segmentação de repetições; métricas por classe e matriz de confusão; experimento controlado de posicionamento de câmera

- **Dados e modelo:** ampliar o dataset com vídeos próprios (amplitude + posicionamento de câmera); extrair dataset direto do MediaPipe, unificando o estimador usado em treino e produção

- **Produto:** novos exercícios; validação de usabilidade com um grupo maior de usuários reais

Conclusão

- App móvel completo de gerenciamento de treinos com um módulo de detecção da qualidade de execução baseado em estimativa de pose e aprendizado de máquina supervisionado

- Cobertura de ponta a ponta: da arquitetura do sistema até a implantação em um ambiente de nuvem real

- Principal lição do desenvolvimento: os maiores riscos técnicos não estão só na escolha do algoritmo de classificação, mas na **representatividade do dataset de treino** frente à variabilidade real de execução dos usuários

Agradecimentos

Ao meu orientador, Prof. Antonio Paulo Salgado Forster, pela orientação ao longo do trabalho;

Aos professores e colegas que acompanharam o desenvolvimento deste projeto;

À minha família, pelo apoio constante.

Obrigado! Perguntas?